E ExitNote

PRODUCT SPECIFICATION / 01

Check once. Remember clearly.

A product requirements document and MVP delivery plan for an offline-first departure confirmation utility.

DOCUMENT	VERSION	PREPARED
PRD + MVP Scope	**1.0 / Working Draft**	**03 November 2026**

NATIVE ANDROID / KOTLIN / JETPACK COMPOSE

OFFLINE-FIRST UTILITY

AT A GLANCE

A small product for a recurring moment.

ExitNote helps people record what they personally checked before leaving home, then retrieve that confirmation record when they start to second-guess themselves.

PRODUCT CATEGORY	FIRST PLATFORM	ARCHITECTURE
Utility	Android	Local-first
Departure confirmation	Native implementation	No account or backend

01 Product brief & principles

02 System & user flows

03 Functional requirements

04 Data model & architecture

05 Non-functional requirements

06 MoSCoW scope

07 Core loop & seven-day plan

08 Acceptance & release criteria

> **Product principle**
> Record the user's confirmation. Never imply that the application independently verified the physical state of a door, appliance, window or device.

PRODUCT OVERVIEW

Make the last check easy to remember.

People sometimes leave home and later wonder whether they locked the door, switched off an appliance, closed a window or collected an essential item. ExitNote gives them a fast way to record those checks and retrieve the record later.

Problem statement

People frequently forget whether they performed routine actions before leaving home. Mental checklists, notes and general-purpose task apps do not always provide a fast, accessible and timestamped record of completed actions.

Product hypothesis

> If people can start a departure check quickly, confirm actions with one tap and retrieve a clear record later, they will rely less on memory alone and may return to the app as part of their departure routine.

Primary job to be done

WHEN / I WANT / SO THAT

"When I am leaving home, help me record the things I have personally checked, so that when I start doubting myself later, I can retrieve what I confirmed without repeating the entire checking process."

Target audience

Primary — The second-guesser

A commuter or household member with a fairly predictable departure routine who sometimes questions whether a door, appliance or window was checked.

Trigger: A few minutes after leaving, the user wonders "Did I actually lock it?"

Secondary — The household manager

A parent or household manager responsible for several departure tasks involving appliances, doors, pets, children and personal essentials.

Trigger: A rushed or multi-person departure with several things to remember.

Secondary — The routine-dependent user

Someone who finds structured sequences helpful and prefers a repeatable checklist. The product must not claim to diagnose, treat or prevent a health condition.

Product positioning

"Check once. Remember clearly. Move on." A focused departure confirmation utility, not a generic habit tracker, task manager or household management platform.

01 / FAST

Start in seconds

One obvious action begins a departure check.

02 / CLEAR

Useful receipts

Every confirmation has a timestamp and retained history.

03 / PRIVATE

Local by default

No account, cloud service or background tracking in MVP.

CORE EXPERIENCE

One short path. One useful receipt.

The product is successful when a user can complete a departure check without navigating a complicated workflow—and can retrieve the last record immediately afterward.

Primary user journey

01	02	03	04	05
Start check	**Confirm**	**Record**	**Finish**	**Retrieve**
Open the app and start a departure session.	Tap each item after completing the action.	Save the confirmation timestamp locally.	Close the session and generate a receipt.	Open the latest receipt when doubt appears.

Flow A — First-time onboarding

1. User installs and opens ExitNote.
2. App explains: “Remember what you checked before leaving.”
3. App creates a preconfigured **Leaving Home** checklist.
4. User reviews and optionally customizes the checklist.
5. User taps **Start Check** and completes the first session.

Target: first useful check within one minute. No account, phone number, email or location permission.

Flow B — Departure check

1. Home screen displays the default checklist.
2. User starts a check; the system creates a new session.
3. Enabled template items are copied into the session as snapshots.
4. User confirms each action; the app immediately saves its timestamp.
5. User finishes the session; the app saves the final state.
6. The latest receipt becomes visible on the home screen.

Flow C — User starts wondering after leaving

1. User wonders whether an action was completed.
2. User opens ExitNote.
3. Home screen immediately shows the latest completed check.
4. User opens the receipt and sees whether the item was confirmed and when.

Flow D — Interrupted session

If the user switches apps, locks the screen, the process is terminated or the device restarts, the unfinished session must remain recoverable.

> **Recovery behaviour:** On the next launch, detect the unfinished session and offer **Resume Check** or **Discard**. Never silently mark outstanding items as complete.

Information architecture

01 / HOME

Start, resume, retrieve

Start Check, unfinished session, latest receipt, checklist settings and history.

02 / ACTIVE CHECK

Confirm actions

Checklist, progress, confirmation state, timestamp and Finish Check.

03 / RECEIPT

Read the record

Session time, per-item confirmations, unconfirmed items and status.

04 / HISTORY

Find previous checks

Completed sessions sorted newest first, each opening its original receipt.

System data path

Every confirmation follows the same deterministic, local-only path. The database commit is the point at which the action becomes a saved record.

STEP	COMPONENT	RESPONSIBILITY
01	Compose UI	User taps a checklist item; the UI sends an intent to its ViewModel.
02	ViewModel	Validates the current session state and invokes the confirmation use case.
03	Use case / repository	Obtains the current instant and timezone, then requests a persistent update.
04	Room transaction	Writes item status and timestamp atomically; failures are returned to the UI.
05	Room Flow → UI	Observed database state updates the screen only after persistence succeeds.

> **Source of truth:** Room is authoritative. Do not treat transient Compose state as a saved confirmation.

BEHAVIOUR SPECIFICATION

Functional requirements.

Requirements are grouped by the product capabilities they enable. “Must” requirements are release blockers for the first MVP.

Epic 01 — Checklist management

Core

ID	REQUIREMENT	ACCEPTANCE CRITERIA
FR-01	Default checklist	First launch creates a “Leaving Home” checklist with sensible starter items.
FR-02	Add items	User can create a custom checklist entry with a non-empty title.
FR-03	Edit items	User can rename a checklist entry; saved changes persist after restart.
FR-04	Delete items	User can remove an item from the active template without altering old receipts.
FR-05	Reorder items	User can change the displayed order; the chosen order persists.
FR-06	Disable items	User can exclude an item from future sessions without deleting it.
FR-07	Persist settings	Checklist changes remain available after application restart.

Starter suggestions: Front door locked; back door locked; stove switched off; oven switched off; windows closed; lights and fans switched off; keys collected; wallet collected; phone collected. These are editable prompts, not mandatory safety checks.

Epic 02 — Check session management

Core

ID	REQUIREMENT	ACCEPTANCE CRITERIA
FR-08	Start session	Creates a unique session and snapshots enabled checklist items.
FR-09	Confirm item	Checking an item saves its current timestamp immediately.
FR-10	Uncheck item	Unchecking clears its active confirmation state and timestamp.
FR-11	Show progress	Displays confirmed item count against the session total.
FR-12	Finish session	Saves completion time and the correct session status.
FR-13	Recover session	An unfinished session remains recoverable after process death or restart.
FR-14	Prevent duplicates	App detects an existing unfinished session before creating another.

> **Timestamp rule:** Repeated taps on an already confirmed item must not overwrite its original timestamp. If the user unchecks and then rechecks it, record a new timestamp for the new confirmation.

BEHAVIOUR SPECIFICATION

Receipts, history & privacy.

Epic 03 — Exit Receipt

Core

ID	REQUIREMENT	ACCEPTANCE CRITERIA
FR-15	Generate receipt	A receipt is available when a session is finished.
FR-16	Show timestamps	Each confirmed item displays its saved confirmation time.
FR-17	Show outstanding items	Unconfirmed items are visually distinct and never presented as completed.
FR-18	Retain history	Past receipts remain available after later sessions are created.
FR-19	Preserve snapshots	Template edits do not alter historical names or item states.
FR-20	Latest receipt	The latest session summary appears on the home screen.

Epic 04 — History

Core

- Display previous sessions in reverse chronological order.
- Show session date, start and completion times.
- Show confirmed and unconfirmed item counts.
- Open a historical session to inspect its original receipt.
- Display a clear empty state when no history exists.

Epic 05 — Privacy & reliability

Core

- No mandatory account or registration.
- No server-side storage or third-party analytics.
- No background location tracking in MVP.
- No unnecessary permissions.
- All session and checklist records remain in private app storage.
- Disclose that uninstalling the app or clearing its data can remove locally stored records unless separately backed up.

Session and item state definitions

SESSION STATUS

IN_PROGRESS

A session that has been started but not finished.

COMPLETED

All session items were confirmed before finishing.

COMPLETED_WITH_UNCONFIRMED

The user explicitly finished while one or more items remained unchecked.

ABANDONED

The user explicitly discarded an unfinished session.

ITEM STATUS

UNCHECKED

No active confirmation exists for this session item.

CHECKED

The user confirmed the item and a timestamp was saved.

Unchecking clears the active checked timestamp. Rechecking creates a new one.

IMPLEMENTATION BLUEPRINT

Local by design.

The first release has no need for authentication, a backend API, cloud database or subscription service. Keep the architecture native, local and easy to inspect.

Recommended stack

LAYER	TECHNOLOGY	RESPONSIBILITY
UI	Kotlin + Jetpack Compose + Material 3	Application screens and user interactions.
State	ViewModel + StateFlow + Coroutines	Observable UI state and asynchronous operations.
Domain	Kotlin use cases	Session creation, confirmation and completion rules.
Data	Repository pattern	Access boundary for local persistence.
Database	Room / SQLite	Templates, items, sessions and historical receipts.
Preferences	DataStore	Small user settings, where needed.
Testing	JUnit + Compose UI tests	Business rules, persistence and critical user journeys.

Logical data relationships

TEMPLATE LAYER

ChecklistTemplate

One template owns many ChecklistItem records. Editing this layer affects future sessions only.

SESSION LAYER

CheckSession

A session stores its start, finish, timezone and status, along with a snapshot of the template name.

TEMPLATE ITEMS

ChecklistItem

Reusable title, icon, sort order and enabled state.

HISTORICAL ITEMS

SessionItem

Copied item title, order, confirmation state and timestamp. This preserves old receipts.

Database schema

```
ChecklistTemplate
- id: String UUID (PK)
- name: String
- isDefault: Boolean
- createdAt: Long
- updatedAt: Long

ChecklistItem
- id: String UUID (PK)
- templateId: String (FK)
- title: String
- icon: String?
- sortOrder: Int
- isEnabled: Boolean
- createdAt: Long

CheckSession
- id: String UUID (PK)
- templateId: String?
- templateNameSnapshot: String
- startedAt: Long
- completedAt: Long?
- timezoneId: String
- status: String

SessionItem
- id: String UUID (PK)
- sessionId: String (FK)
- sourceItemId: String?
- titleSnapshot: String
- sortOrder: Int
- status: String
- checkedAt: Long?
- checkedTimezoneId: String?
```

Use UUID primary keys. Store timestamps as UTC epoch milliseconds and retain timezone identifiers for historically accurate display. Define explicit Room foreign keys and deletion policies.

Data integrity rules

Atomic session start

Create the session and all snapshot items in one database transaction. Do not expose a partial session.

Atomic confirmation

Update item status, timestamp and timezone together. If the write fails, retain the prior state.

Completion consistency

Count item states and save completion time and session status within one transaction.

Historical preservation

Template deletion must not cascade into completed sessions. Keep snapshots independent of templates.

Useful indexes: CheckSession(status), CheckSession(startedAt DESC), SessionItem(sessionId, sortOrder). Index only fields used for actual lookup and ordering.

RELIABILITY CONTRACT

Make the record dependable.

The user is relying on the receipt as a memory aid. A lost or misleading record undermines the entire product proposition.

AREA	REQUIREMENT	IMPLEMENTATION NOTE
Performance	Fast cold launch and immediate feedback.	Keep the home screen lean; avoid network dependencies.
Offline	All core flows work without internet.	Do not add INTERNET permission or remote services.
Persistence	Commit changes before showing success.	Room database is the source of truth.
Recovery	Unfinished session survives process death.	Persist each confirmation immediately.
Security	Keep records inside private app storage.	No public database or shared storage exposure.
Privacy	Collect no unnecessary personal data.	No account, tracking, ads or third-party analytics.
Accessibility	Large touch targets and screen-reader labels.	Use semantic Compose components and scalable text.
Compatibility	Support a sensible Android version range.	Choose minSdk after inspecting current project constraints.
Maintainability	Clear separation of UI, logic and data.	Avoid overengineering and unnecessary modules.

Trust and wording rules

Preferred language

“You confirmed this at 8:42 AM.”

“This item has not been confirmed in this session.”

Prohibited implication

Do not say “Your door is verified locked”, “Your stove is safe” or imply a physical device has been sensed. ExitNote records user input only.

Important edge cases

- Database write fails: show an error and do not show a saved confirmation.
- Checklist is empty or all items are disabled: provide a clear recovery path; do not create a misleading empty receipt.
- Template is deleted: historical sessions and their snapshots remain readable.
- Device timezone changes: preserve saved instants and timezone identifiers for display.
- Repeated item tap: do not overwrite the original timestamp unless the user unchecked and rechecked.
- Duplicate session start: detect and offer to resume the existing unfinished session.
- App is terminated mid-check: recover the unfinished session without marking it complete.

Release 1 exclusions

- Smart-lock integration
- Automatic appliance verification
- Background GPS or geofencing
- AI or voice assistant features
- Social or community features
- Household cloud synchronization
- Health diagnosis or treatment claims
- Subscription infrastructure

SCOPE CONTROL

Ship the smallest complete utility.

MoSCoW separates what must exist for a useful first release from enhancements that can wait. The first version should complete the full confirmation-to-receipt loop.

Must Have

Release blocker

- First-run default checklist.
- Start departure check.
- Check and uncheck items.
- Automatic timestamps.
- Finish check and generate receipt.
- Latest receipt on home screen.
- Local persistent database.
- Previous session history.
- Basic checklist customization.
- Interrupted-session recovery.
- Clear confirmed / unconfirmed states.

Should Have

Next iteration

- Multiple checklist templates.
- Optional local daily reminder.
- Home-screen widget.
- Light and dark themes.
- Haptic feedback.
- JSON export and import.

Could Have

Future experiments

- Photo attachments.
- Verbal confirmation.
- Quick Settings / shortcut integration.
- Family checklist sharing.
- Optional encrypted backup.
- Premium customization.

Won't Have

Out of scope

- Background location tracking.
- Automatic smart-home verification.
- AI-powered checking.
- Health or mental-health treatment features.
- Social feeds.
- Cloud accounts and subscriptions in v1.

Explicit MVP boundary

BUILD NOW

Start → Confirm → Timestamp → Receipt → Retrieve

If a proposed feature does not strengthen this loop or protect its data integrity, it does not belong in the first release.

Monetization hypothesis

TIER	PROPOSED OFFER	PURPOSE
Free	One checklist, timestamps, receipts and basic history.	Validate real repeated usage without payment friction.
Pro — one-time	Test ₹149–299 for unlimited templates and customization.	Test willingness to pay for expanded utility.
Annual — later	Test ₹199–399/year only if continuing services justify it.	Consider only after backups, sync or other ongoing value exists.

These are experiment prices, not validated optimal prices or revenue forecasts. A simple offline application has a weaker subscription rationale without continuing premium services.

DELIVERABLE SLICE

The thinnest complete slice.

The first build should be a usable Android app that completes the core loop reliably. It is not a prototype with several screens that do not persist data.

INPUT

User starts a departure check

The app loads enabled checklist items into a new session snapshot.

ACTION

User confirms each task

Each confirmation is immediately committed to Room with a timestamp.

OUTPUT

Exit Receipt is created

The receipt shows confirmed and unconfirmed items and their recorded times.

RETURN VALUE

User retrieves the latest receipt

The home screen surfaces the latest record without requiring history navigation.

Seven-day development plan

- **Day 01 — Project foundation**
  Inspect repository, verify Gradle and Kotlin compatibility, set up Compose, theme, navigation and application structure.
- **Day 02 — Database**
  Implement four Room entities, DAOs, repositories, database initialization, UUIDs, default checklist seeding and persistence tests.
- **Day 03 — Checklist interface**
  Build Home and checklist editing: add, rename, delete, reorder and disable entries.
- **Day 04 — Session engine**
  Implement session creation, item snapshots, check / uncheck, timestamp persistence, completion rules and recovery.
- **Day 05 — Receipt and history**
  Implement receipt generation, per-item times, unconfirmed states, history list and latest receipt on Home.
- **Day 06 — Polish and tests**
  Implement empty, error and confirmation states; accessibility, responsive layout, themes where practical and automated tests.
- **Day 07 — Device validation**
  Test on a physical device, fix bugs, verify offline operation, create a debug APK and document release build steps.

> **Time-boxing rule:** Seven days is a delivery target for a narrow solo-developer MVP, not a guarantee. If a critical dependency or build issue appears, preserve the core loop and defer nonessential polish.

VALIDATION CHECKLIST

Complete means dependable.

The MVP is ready when the complete user journey works, the underlying record is trustworthy, and the app remains useful without a network connection.

Release acceptance criteria

TEST	EXPECTED RESULT
First launch	Default “Leaving Home” checklist is created once and displayed without registration.
Start check	Creates a unique session and copies enabled item names into snapshots.
Confirm item	Timestamp is saved; UI reflects the saved state only after successful persistence.
Repeated tap	Original timestamp remains unchanged.
Uncheck / recheck	Active confirmation clears, then a new timestamp is saved on recheck.
Partial completion	Outstanding items remain explicitly unconfirmed after the user finishes.
Process death	Unfinished session can be recovered after forced app termination and relaunch.
Template edit	Previously completed session receipts remain unchanged.
Offline	All core screens, session operations and history work without internet.
Permissions	No unnecessary permission is requested by the MVP.
Build	Debug APK builds successfully and installs on a physical Android device.

Suggested early product metrics

These are initial experiment thresholds for learning, not external industry benchmarks.

ACTIVATION

< 60s

Target time to first useful check.

INTERACTION

< 30s

Target time for a typical check.

WEEK 1

30%+

Proposed second-session target within seven days.

WEEK 2

20%+

Proposed return target in week two.

CORE VALUE

Receipts

Track how often users retrieve a previous receipt.

Deferred roadmap

RELEASE	CANDIDATE CAPABILITY	GATE
v1.1	Multiple templates, local reminder, widget, haptic feedback, JSON export / import.	Core loop works and users return.
v1.2	Photo attachments, verbal confirmation, shortcut integration.	Users demonstrate demand for richer proof.
Later	Encrypted backup, family sharing, optional synchronization, possible paid tier.	Clear recurring value and a justified privacy model.
Separate research	Geofencing and smart-home integration.	Prove that automation materially improves the product and assess platform / policy costs.

PRODUCT NORTH STAR

One complete check. One useful record.

Prioritize correctness, speed, privacy and a dependable core loop over feature quantity. Build a small product that works end to end.

Decisions before implementation

01 / AUDIENCE

Choose a narrow first user

Start with commuters and household members who repeatedly second-guess departure checks.

02 / PROMISE

Promise a record, not verification

The app stores explicit user confirmations; it does not sense the state of physical devices.

03 / SCOPE

Keep version one local

Avoid accounts, location permissions, cloud sync and monetization infrastructure until usage is demonstrated.

Go / no-go validation gates

GATE	EVIDENCE REQUIRED	DECISION
Problem frequency	Target users report recent, repeated departure-related doubt or forgetfulness.	Continue only if the problem recurs in the target segment.
Repeat behaviour	Users voluntarily complete multiple sessions and retrieve receipts after leaving.	Prioritize retention evidence over feature requests.
Payment signal	Users demonstrate willingness to pay for a specific premium benefit.	Do not add a subscription without ongoing value.

Working product name: ExitNote. Name, trademark, domain and app-store availability have not been cleared. This document describes the proposed product and implementation scope; it is not evidence of commercial validation.